TECHNICAL REQUIREMENTS DOCUMENT

AVR-Map — Adaptive Variable-Resolution Semantic Multi-Surface 2.5D LiDAR Mapping System

SIH Problem Statement: SIH26053
Organisation: DRDO
Document Type: Technical Requirements Document — TRD
Prototype: AVR-Map V1
Version: 1.0
Date: September 2026
Status: Implementation Baseline

---

1. Purpose of this TRD

This Technical Requirements Document defines **how the AVR-Map prototype shall actually be engineered**.

The PRD answers:

> What are we building and why?

This TRD answers:

> Exactly how will the system be structured, what algorithms will it use, what data moves between components, what performance must be achieved, and how will we prove that it works?

This document is intended to become the technical source of truth for:

- software architecture;
- ROS 2 package development;
- map-engine implementation;
- AI integration;
- dataset integration;
- algorithms;
- data structures;
- threading;
- memory management;
- configuration;
- visualisation;
- benchmarking;
- unit/integration/system testing;
- deployment;
- future technical documents.

---

2. Technical Objective

The prototype shall transform sequential 3D LiDAR scans into a:

> **Sparse hierarchical semantic multi-surface 2.5D local map whose spatial resolution adapts according to distance and safety-relevant information.**

The system shall answer, in real time:

```
Where is terrain?

What is its elevation?

What objects exist?

Which objects are moving?

Which areas are traversable?

What vertical clearance exists?

How confident is the perception?

What spatial resolution is necessary here?

How much memory/computation is being saved?
```

The central runtime transformation is:

```
Raw 3D LiDAR
      ↓
Semantic + geometric understanding
      ↓
Importance assessment
      ↓
Adaptive spatial allocation
      ↓
Hierarchical multi-surface 2.5D map
```

---

3. Requirement Language

The following terminology is normative.

SHALL / MUST
Mandatory for AVR-Map V1.

SHOULD
Strongly recommended, but may be relaxed if implementation constraints require it.

MAY
Optional enhancement.

V2
Explicitly excluded from the critical V1 path.

---

4. Source Hierarchy

This TRD uses three kinds of information.

4.1 SIH / research-derived requirements

Examples:

- raw LiDAR input;
- semantic terrain and object perception;
- non-uniform 2.5D representation;
- near-field detail;
- far-field coarsening;
- boundary consistency;
- real-time execution;
- FPS/accuracy/memory evaluation.

The research dossier decomposes the technical problem into pose compensation, semantic segmentation, motion estimation, terrain geometry, vertical geometry, adaptive resolution, hierarchical indexing, traversability and complete-pipeline real-time performance.

4.2 Research-supported engineering principles

Examples:

- hierarchical mapping;
- multi-layer 2.5D maps;
- Morton/Z-order indexing;
- uncertainty-aware semantic perception;
- Kalman/Hungarian tracking;
- rolling local maps.

ANYbotics `grid_map`, for example, provides multi-layer 2.5D grid maps and an efficient circular-buffer mechanism for moving maps, validating several architectural principles reused here.

4.3 AVR-Map design decisions

Examples:

```
5 / 10 / 20 / 40 cm hierarchy

16 × 16-cell tiles

2–5 scan replay cache

specific ROS topic names

specific hysteresis rules

specific benchmark modes
```

These are **prototype engineering decisions**, not claims that literature proves these are globally optimal.

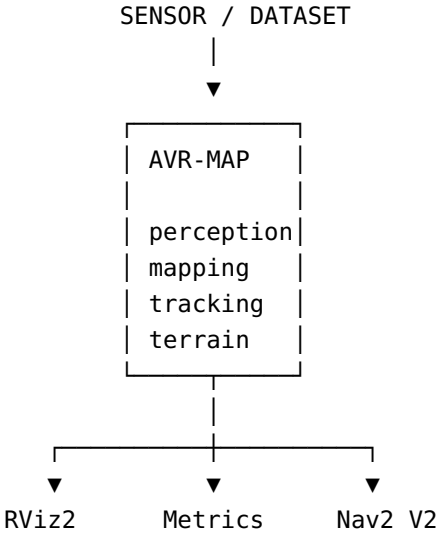
They remain configurable.

---

5. System Context

AVR-Map is **not** a complete autonomous vehicle.

The prototype sits between perception sensors/datasets and navigation.



AVR-Map SHALL NOT implement full SLAM.

Vehicle pose shall be obtained from:

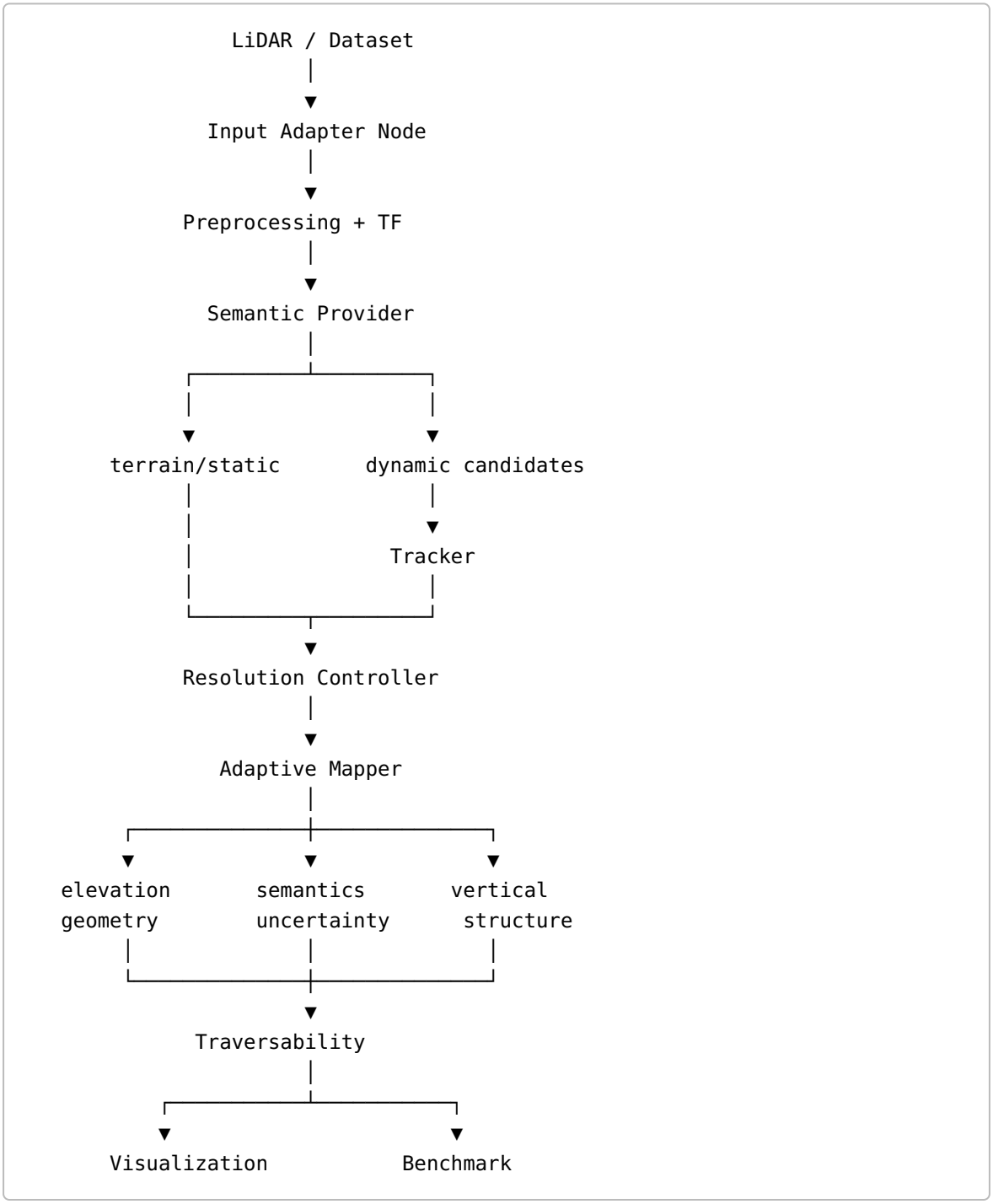
- dataset poses;
- odometry;
- existing SLAM;

• simulator;
• ground-truth pose stream.

---

6. Prototype Architecture Decision

The prototype shall use the following architecture:



---

7. Chosen Development Platform

7.1 Primary development environment

AVR-Map V1 SHALL target:

```
OS:
Ubuntu 24.04 LTS

ROS:
ROS 2 Jazzy Jalisco

Architecture:
x86-64

Compiler:
GCC / Clang with C++17 minimum

Build:
colcon + ament_cmake

Python:
3.x version supported by Ubuntu/ROS environment
```

ROS 2 Jazzy officially supports Ubuntu 24.04 on both x86-64 and ARM64.

---

8. Why ROS 2 Jazzy

Jazzy is selected because:

- modern ROS 2 distribution;
- Ubuntu 24.04 support;
- RViz2 availability;
- tf2;
- rosbag2;
- Nav2 compatibility;
- active ecosystem;
- maintained `grid_map` Jazzy branch.

`grid_map` currently maintains ROS 2 Humble, Iron, Jazzy and Rolling branches.

---

9. Supported Hardware

9.1 Development workstation

Recommended:

```
CPU:
6+ modern cores

RAM:
16 GB minimum
32 GB preferred

GPU:
NVIDIA CUDA-capable GPU

VRAM:
8 GB preferred

Storage:
100+ GB available for datasets
```

9.2 Minimum mapping-only system

When using ground-truth semantic labels instead of neural inference:

```
CPU:
4+ cores

RAM:
16 GB

GPU:
not mandatory
```

9.3 Edge deployment

Jetson Orin-class deployment is:

```
V1 Stretch Goal
```

not a mandatory completion criterion.

---

10. Mac Development Policy

macOS MAY be used for:

- documentation;
- code editing;
- Git;
- algorithm prototyping;
- dataset inspection;
- UI development.

The official runtime environment SHALL remain Ubuntu Linux because:

- ROS support;
- CUDA;
- TensorRT;
- NVIDIA tooling;
- LiDAR drivers;
- robotics ecosystem compatibility.

---

11. Programming Language Allocation

C++

Mandatory for:

```
adaptive mapping
spatial indexing
cell aggregation
rolling-map management
traversability
benchmark instrumentation
```

Reason:

These are latency-sensitive and memory-sensitive operations.

Python

Preferred initially for:

```
semantic neural inference
dataset conversion
offline analysis
```

```
plotting
benchmark evaluation
```

The semantic module MUST expose a stable interface so implementation language can later change without affecting mapping.

---

12. Core Technology Stack

```
ROS 2 Jazzy
C++17+
Python
PyTorch
PCL
Eigen
tf2
RViz2
Foxglove optional
rosbag2
NumPy
Open3D offline/diagnostic use
```

---

13. AI Runtime

Initial runtime:

```
PyTorch
```

Future deployment interfaces SHOULD support:

```
ONNX Runtime
TensorRT
```

TorchSparse++ MAY be evaluated if a sparse-convolution model such as MinkUNet/SPVCNN replaces the initial semantic provider.

TorchSparse++ is a legitimate GPU sparse-convolution framework, but framework-level speedups SHALL NOT be assumed to equal complete AVR-Map pipeline speedups.

---

14. Semantic Model Strategy

The semantic system SHALL use a provider abstraction:

```
class SemanticProvider
{
public:
    virtual SemanticCloud infer(
        const PointCloud& input
    ) = 0;
};
```

Implementations:

```
GroundTruthSemanticProvider

SalsaNextSemanticProvider

FutureSparseCNNProvider
```

---

15. Why Ground-Truth Provider Is Mandatory

Mapping development MUST NOT depend on ML integration.

Ground-truth mode allows us to isolate:

```
map correctness

resolution logic

small-object preservation

boundary correctness

memory

latency
```

without semantic model error contaminating the result.

This enables two separate questions:

```
Does our map engine work?

Does the complete perception system work?
```

instead of confusing them.

---

16. Initial Neural Semantic Provider

Preferred V1 neural baseline:

SalsaNext

Reason:

- LiDAR semantic segmentation;
- range-image approach;
- pretrained SemanticKITTI support;
- designed for real-time operation;
- uncertainty-aware formulation;
- relatively straightforward semantic output.

The official implementation describes SalsaNext as a fast uncertainty-aware LiDAR semantic segmentation architecture and includes epistemic/aleatoric uncertainty estimation.

---

17. Semantic Model Is Replaceable

The architecture SHALL NOT hard-code SalsaNext assumptions.

Future candidates:

```
MinkUNet
SPVCNN
Cylinder3D
SphereFormer
WaffleIron
```

shall be able to produce the same standardized semantic output.

---

18. Dataset Requirements

Mandatory development datasets:

SemanticKITTI

Primary use:

```
urban semantics
moving/static label experiments
semantic model baseline
```

RELLIS-3D

Primary use:

```
off-road terrain
rough terrain
vegetation
unstructured environments
```

RELLIS-3D contains 13,556 annotated LiDAR scans, 20 classes including void, Ouster/Velodyne data, poses, ROS bags and off-road environments.

Optional:

```
GOOSE
SemanticPOSS
nuScenes
WildScenes
own rosbag
```

---

19. Standard Coordinate Frames

AVR-Map SHALL use:

```
map            optional global frame

odom           continuous local frame

base_link      vehicle frame

lidar          sensor frame
```

Core mapping frame:

```
odom
```

---

20. Frame Relationship

```
odom
  │
  └── base_link
        │
        └── lidar
```

All incoming LiDAR points SHALL be transformed from:

```
lidar
```

to:

```
odom
```

before persistent map insertion.

---

21. Why Odom Frame Is Used

The local adaptive map should remain continuous.

A global SLAM `map` transform may change following loop closure.

Therefore:

```
real-time AVR map
→ odom frame

future persistent/global representation
→ map frame
```

---

22. ROS Node Architecture

Required ROS packages:

```
avr_msgs
avr_input
avr_preprocess
avr_semantics
avr_mapping
avr_tracking
avr_traversability
avr_visualization
avr_benchmark
avr_bringup
```

---

23. Input Node

Package

```
avr_input
```

Responsibilities

- dataset playback;
- rosbag playback integration;
- live PointCloud2 forwarding;
- dataset pose extraction;
- timestamp validation.

Input

Dataset or sensor.

Output

```
/avr/points_raw
/avr/odometry
```

---

24. Input Point Cloud Interface

ROS message:

```
sensor_msgs/msg/PointCloud2
```

Minimum fields:

```
x float32
y float32
z float32
```

Preferred:

```
intensity
ring
time
```

---

25. Odometry Interface

ROS message:

```
nav_msgs/msg/Odometry
```

ROS defines `nav_msgs/Odometry` as an estimate of position and velocity in free space.

Topic:

```
/avr/odometry
```

---

26. Preprocessing Node

Package

```
avr_preprocess
```

Input

```
/avr/points_raw
/avr/odometry
/tf
```

Output

```
/avr/points_filtered
```

Responsibilities

1. validate cloud;
2. remove NaN/Inf;
3. crop to ROI;
4. remove sensor-internal points;
5. transform to desired frame;
6. optional deskew;
7. preserve original point index where possible;
8. generate point-range information.

---

27. Region of Interest

Default:

```
horizontal radius:
100 m

z_min:
configurable

z_max:
configurable
```

ROI values SHALL depend on dataset/sensor.

No point farther than active map radius needs to reach mapping.

---

28. Deskew Requirement

If per-point timing and motion data exist, preprocessing SHOULD compensate for motion occurring during a LiDAR rotation.

V1 SHALL support:

```
deskew_enabled: true/false
```

If timing information is unavailable:

```
frame treated as instantaneous
```

and benchmark documentation must state this limitation.

---

29. Semantic Point Format

The semantic node shall output:

```
sensor_msgs/PointCloud2
```

with fields:

```
x              float32
y              float32
z              float32
intensity      float32
semantic_label uint16
confidence     float32
uncertainty    float32
```

All fields correspond to one point.

---

30. Semantic Uncertainty

When full class probability vector:

```
p1 ... pC
```

is available, normalized entropy MAY be used:

```
H = - Σ p(c) log p(c)

U = H / log(C)
```

Then:

```
0 ≤ U ≤ 1
```

where:

```
0 → confident distribution

1 → highly uncertain distribution
```

If only top-class probability exists:

```
uncertainty = 1 - confidence
```

is acceptable in V1.

---

31. Mapping Core Overview

Package:

```
avr_mapping
```

This is the principal AVR-Map component.

It SHALL implement:

```
adaptive resolution controller

hierarchical cell ownership

Morton indexing

tile allocation

split

merge

cell fusion

multi-surface geometry

semantic fusion

staleness

rolling-map pruning

point replay cache
```

---

32. Resolution Hierarchy

Base resolution:

```
r0 = 0.05 m
```

Levels:

```
L0 = 0.05 m
L1 = 0.10 m
L2 = 0.20 m
L3 = 0.40 m
```

General formula:

```
r(level) = r0 × 2^level
```

Maximum V1:

```
level = 3
```

---

33. Why Dyadic Resolution

Using:

```
5
10
20
40 cm
```

instead of arbitrary:

```
5
15
25
50 cm
```

provides exact nesting.

Research specifically recommends powers-of-two multiples because coarse/fine boundaries align automatically.

---

34. Baseline Distance Policy

Initial V1:

```
0 ≤ d < 10 m       → L0

10 ≤ d < 30 m      → L1

30 ≤ d < 60 m      → L2

60 ≤ d ≤ 100 m     → L3
```

This is not claimed as optimal.

All thresholds SHALL be configurable.

---

35. Resolution Controller

Resolution is defined as:

```
L_final =
clamp(
    L_distance
    - semantic_refinement
    - uncertainty_refinement
    - dynamic_refinement
    - geometry_refinement
)
```

where lower levels correspond to finer cells.

---

36. Resolution Controller Pseudocode

```
function choose_level(point, context):

    level = distance_level(point.range)

    level -= semantic_refinement(point.class)
```

```
    if sufficient_density(context):
        level -= uncertainty_refinement(
            point.uncertainty
        )

    if context.dynamic:
        level -= dynamic_refinement

    if context.geometry_complex:
        level -= geometry_refinement

    return clamp(level, L0, L3)
```

---

37. Semantic Refinement Table

Initial configuration:

```
pedestrian      → refine 2 levels
cyclist         → refine 2
pole            → refine 2
small barrier   → refine 2
traffic cone    → refine 2

vehicle         → refine 1
large obstacle  → refine 1
curb            → refine 1 or 2

road            → 0
flat terrain    → 0
building        → 0
vegetation      → 0 initially
```

Mappings MUST be dataset-specific.

---

38. Semantic Refinement Does Not Override Sensor Information

A far pedestrian with three LiDAR returns cannot magically become a detailed 5 cm object.

Therefore every refinement request SHALL pass:

```
information availability gate
```

before subdivision.

---

39. Point Density Gate

For a local neighbourhood:

```
ρ = number_of_points / area
```

Refinement is allowed only if:

```
ρ ≥ configured minimum
```

or another equivalent evidence test succeeds.

Alternative acceptable test:

```
number of recent raw points inside parent >= Nmin
```

---

40. Uncertainty Refinement

If:

```
U >= uncertainty_threshold
```

AND:

```
sufficient observations exist
```

then:

```
refine one level
```

Otherwise:

```
mark uncertain/unknown
```

The system SHALL NEVER assume that increasing map resolution creates new sensor information.

---

41. Dynamic Refinement

Confirmed dynamic tracks SHOULD enforce:

```
maximum L1 = 10 cm
```

where information density permits.

Example:

```
70 m moving vehicle

base:
40 cm

dynamic override:
10 cm
```

subject to point-density gate.

---

42. Geometry Refinement

A cell may refine if:

```
height variance high

large neighbour step

roughness high

multiple vertical surfaces

small geometric cluster detected
```

This provides protection against semantic failures.

---

43. Refinement Priority

Recommended ordering:

```
1. physical observation sufficiency

2. hard geometric hazard

3. confirmed dynamic object

4. critical semantic class

5. uncertainty

6. distance baseline
```

Distance remains baseline, but safety signals may override it.

---

44. Split Operation

When a cell at level:

```
L
```

must refine:

```
parent L
      ↓
4 children at L-1
```

Example:

```
20 cm
      ↓
4 × 10 cm
```

The parent becomes:

```
non-leaf
```

and SHALL NOT directly own measurements.

---

45. Split Replay Requirement

After a split, child data SHALL NOT be generated by blindly copying the parent's aggregate.

Recent raw points SHALL be replayed.

```
Parent
  ↓ split
Point Replay Buffer
  ↓
re-bin observations
  ↓
Child statistics
```

This is essential to recover the spatial detail that triggered refinement.

---

46. Point Replay Buffer

Each tile SHOULD retain approximately:

```
2–5 recent scans
```

of point observations relevant to it.

Recommended V1:

```
3 scans
```

The exact memory cost SHALL be benchmarked.

---

47. Merge Operation

Four sibling leaves MAY merge into their parent if:

```
no hazard active

no dynamic object active

uncertainty below threshold

geometry sufficiently homogeneous
```

```
minimum stable-frame count reached
```

---

48. Coarsening Hysteresis

Refinement:

```
fast
```

Coarsening:

```
slow
```

Recommended initial behaviour:

```
refine after:
1 qualifying observation

coarsen after:
10 stable observations
```

These values are tuning parameters, not safety guarantees.

---

49. Active Leaf Invariant

Exactly one leaf resolution shall own a spatial region.

```
parent active
OR
children active
```

Never both.

The research specifically identifies one-active-leaf ownership as necessary to avoid double-counting observations.

---

50. Coordinate Quantisation

Let:

```
r0 = finest resolution
```

For map coordinate:

```
(x, y)
```

finest-grid integer coordinates are:

```
bx = floor((x - origin_x) / r0)

by = floor((y - origin_y) / r0)
```

At hierarchy level `L`:

```
scale = 2^L

cx = floor_div(bx, scale)

cy = floor_div(by, scale)
```

Integer quantisation occurs once.

---

51. Half-Open Boundaries

Every cell SHALL represent:

```
[x_min, x_max)

[y_min, y_max)
```

Using half-open intervals removes ambiguous edge ownership.

A point exactly on:

```
x_max
```

belongs to the next cell.

---

52. Negative Coordinate Handling

Native C++ integer division truncates toward zero.

For negative map coordinates this can cause incorrect indexing.

AVR-Map SHALL implement mathematically correct:

```
floor_div()
```

rather than relying on `/` for signed values.

This shall be unit-tested.

---

53. Morton Encoding

Spatial tiles shall use Morton/Z-order encoding.

Given unsigned integer coordinates:

```
x
y
```

their bits are interleaved:

```
x2 y2 x1 y1 x0 y0
```

to form:

```
morton_xy
```

---

54. Signed-to-Unsigned Encoding

Before Morton interleaving, signed tile indices SHOULD use:

```
zig-zag encoding
```

or an equivalent deterministic bias.

Example:

```
0  → 0
-1 → 1
1  → 2
-2 → 3
2  → 4
```

This avoids wasting half the key space.

---

55. Tile Structure

The hierarchy shall allocate tiles rather than individual heap objects.

Recommended:

```
16 × 16 cells per tile
```

Logical tile key:

```
struct TileKey
{
    uint8_t level;
    uint64_t morton_xy;
};
```

---

56. Why Tiles

Tiles provide:

- fewer hash entries;
- better cache locality;
- contiguous cell iteration;
- simpler serialization;
- easier GPU migration;
- easier pruning;
- less allocator overhead.

---

57. Map Container

Prototype container:

```
std::unordered_map<TileKey, Tile>
```

or a high-performance compatible hash map.

V1 correctness takes priority over specialized hash-table micro-optimisation.

Benchmarking MAY later compare:

```
std::unordered_map
robin-hood hashing
flat hash tables
```

---

58. Cell Internal Representation

Recommended logical cell:

```
struct Cell
{
    float ground_mean;
    float ground_variance;

    float z_min;
    float z_max;

    float overhang_low;
    float overhang_high;

    float roughness;

    float occupancy_log_odds;

    float semantic_confidence;
    float hazard_confidence;

    float uncertainty;

    float traversability;

    float dynamic_probability;
```

```
    uint32_t last_seen_ms;

    uint16_t point_count;

    uint16_t semantic_label;
    uint16_t hazard_label;

    uint8_t resolution_level;
    uint8_t flags;
};
```

The actual packed size SHALL be obtained using:

```
sizeof(Cell)
```

and reported in the benchmark.

No theoretical byte figure shall be presented as measured memory.

---

59. Flags

Suggested bit flags:

```
bit 0 → VALID_GROUND
bit 1 → OCCUPIED
bit 2 → OVERHANG
bit 3 → DYNAMIC
bit 4 → UNCERTAIN
bit 5 → SECONDARY_HAZARD
bit 6 → UNKNOWN
bit 7 → RESERVED
```

---

60. Hot/Cold Data Optimisation

V1 MAY initially use one cell structure.

If profiling shows memory pressure, split into:

```
HOT

geometry
occupancy
label
```

```
confidence
timestamps
```

and:

```
COLD

debug information
full class probabilities
extended statistics
```

Only hot data should be touched every frame.

---

61. Current-Frame Point Aggregation

The map SHALL first group current-frame point observations by active cell.

For each cell calculate:

```
point_count
z_min
z_max
mean
variance
semantic evidence
hazard evidence
```

---

62. Robust Ground Observation

For ground-candidate points:

If:

```
N >= minimum percentile count
```

compute:

```
z_ground_obs ≈ low robust percentile
```

such as:

```
P10
```

or median of lowest valid surface group.

For sparse cells:

```
minimum / robust minimum
```

may be used with higher uncertainty.

---

63. Ground Fusion

Maintain:

```
ground_mean
ground_variance
```

Using a 1D probabilistic update.

Given prior:

```
μp, σp²
```

measurement:

```
zm, σm²
```

Kalman gain:

```
K = σp² / (σp² + σm²)
```

Updated mean:

```
μ = μp + K(zm - μp)
```

Updated variance:

```
σ² = (1-K)σp²
```

Large inconsistent jumps shall be classified as potential obstacle/step instead of blindly fused.

---

64. Height Variance

For observations in a cell, Welford's online algorithm SHOULD be used for stable streaming variance calculation.

Outputs support:

```
roughness

mixed-surface detection

geometry refinement
```

---

65. Occupancy Update

Occupancy SHOULD use log-odds.

```
L_t =
L_(t-1)
+
inverse_sensor_update
-
prior
```

Clamp:

```
Lmin ≤ L ≤ Lmax
```

V1 does not require complex ray tracing for every point if performance is insufficient.

Minimum:

```
hit evidence
+
temporal decay
```

---

66. Semantic Fusion

Do NOT implement:

```
latest prediction wins
```

or simple permanent majority vote.

Maintain weighted semantic evidence.

For observation class `c`:

```
E_c(t) =
λ E_c(t-1)
+
w × p(c)
```

where:

```
λ = temporal retention/decay
w = confidence/range weighting
```

---

67. Compact Semantic State

V1 should avoid storing the full class vector in every cell.

Store:

```
dominant_label
dominant_confidence

hazard_label
hazard_confidence
```

This preserves small safety-important minority classes without full C-dimensional storage.

---

68. Secondary Hazard Rule

Example:

```
95 road points
3 pole points
2 curb points
```

Result must not be simply:

```
road
```

Instead:

```
dominant_label = road

hazard_label = pole

hazard_confidence = non-zero

SECONDARY_HAZARD = true
```

This may trigger local refinement.

---

69. Small-Object Preservation Pipeline

```
semantic/geometric minority evidence
            ↓
secondary hazard
            ↓
refinement request
            ↓
point-density gate
            ↓
split
            ↓
recent-point replay
            ↓
fine child cells
```

This is one of the main AVR-Map innovations.

---

70. Multi-Surface Requirement

Ordinary one-height elevation maps cannot represent:

```
bridge above road

tree branch above terrain

overhead sign

boom barrier

tunnel roof
```

The research explicitly identifies multilevel environments as a structural limitation of single-height 2.5D mapping.

Therefore AVR-Map SHALL maintain lightweight vertical-surface information.

---

71. Vertical Profile Processing

During each frame, points within a cell MAY temporarily be grouped into vertical bins.

Example:

```
10 cm Z bins
```

This is temporary computation.

It does NOT mean persistent full 3D voxels.

---

72. Ground-Connected Obstacle

If vertical returns continuously extend from approximately:

```
ground_z
```

upwards:

```
ground
███
███
███
```

the structure is treated as:

```
ground-connected obstacle
```

and the cell is blocked.

---

73. Overhang Candidate

If:

```
valid ground exists

AND

elevated returns exist

AND

vertical free gap exists
```

then:

```
overhang_low
```

is set to the lowest reliable overhead surface.

---

74. Vertical Clearance

```
clearance =
overhang_low - ground_mean
```

If no overhang:

```
clearance = +∞ / unknown
```

depending on observation status.

---

75. Clearance Traversability Veto

Given:

```
vehicle_height
safety_margin
```

if:

```
clearance
<
vehicle_height + safety_margin
```

then:

```
traversable = false
```

---

76. Dynamic Candidate Extraction

Dynamic-capable semantic classes:

```
person
cyclist
car
truck
motorcycle
```

depend on dataset ontology.

Candidate points SHALL be clustered after ego-motion compensation.

---

77. Clustering

V1 algorithms:

```
DBSCAN
```

or:

```
Euclidean clustering
```

Preferred initial implementation:

```
DBSCAN
```

because it:

- does not require known cluster count;
- naturally removes isolated noise;
- works with irregular point distributions.

Parameters MAY vary with range.

---

78. Cluster Representation

Each detection:

```
struct Detection
{
    uint16_t semantic_class;

    Vec3 centroid;

    Vec3 dimensions;

    BoundingBox3D box;

    float confidence;

    uint32_t point_count;
};
```

---

79. Tracker State

Constant-velocity state:

```
[x
 y
 z
 vx
 vy
 vz]
```

Optional box dimensions stored separately.

---

80. Kalman Motion Model

State transition for time step `dt`:

```
x' = x + vx dt
y' = y + vy dt
z' = z + vz dt

vx' = vx
vy' = vy
vz' = vz
```

---

81. Data Association

Tracker-detection cost may combine:

```
centroid distance

3D IoU

class mismatch penalty
```

Associations SHALL be solved using:

```
Hungarian algorithm
```

This mirrors the simple real-time design philosophy of AB3DMOT, whose official implementation uses a 3D Kalman filter plus Hungarian data association.

---

82. Track Lifecycle

Recommended initial parameters:

```
tentative track:
until 3 successful matches

confirmed:
after 3 matches

max misses:
5 frames
```

```
dynamic:
speed exceeds configured threshold
for multiple observations
```

Values SHALL be configurable.

---

83. Static vs Dynamic Separation

Semantic category is not motion state.

```
car ≠ necessarily moving

person ≠ necessarily moving
```

Motion must be inferred temporally.

---

84. Dynamic Map Layer

Confirmed moving tracks SHALL contribute to:

```
short-lived dynamic occupancy
```

not permanent terrain occupancy.

TTL:

```
short
```

e.g.:

```
0.5–2 seconds
```

depending on update rate.

---

85. Stationary Object Promotion

A previously dynamic-capable object MAY be integrated into static occupancy only if:

```
velocity remains below threshold

for sustained duration

with stable spatial observations
```

This is optional V1 behaviour.

---

86. Traversability Package

```
avr_traversability
```

Input:

```
adaptive map leaves
vehicle profile
```

Output:

```
traversability layer
```

---

87. Vehicle Profile

Configuration:

```
vehicle:
  width_m:
  height_m:
  ground_clearance_m:
  max_step_m:
  max_slope_deg:
  safety_margin_m:
```

No universal values shall be invented.

---

88. Terrain Neighbourhood Query

For each target cell:

```
query neighbouring active leaves
within configured metric radius
```

Because neighbouring cells can differ in resolution, algorithms shall operate in **metric space**, not assume one fixed grid index step.

---

89. Coarse/Fine Boundary Handling

For calculations such as slope:

```
fine cells next to coarse cell
```

shall be handled using one of:

```
weighted cell-center samples

or

coarse footprint expanded into equivalent fine footprint
```

V1 recommended:

> gather all active leaf ground samples inside a metric neighbourhood and fit terrain geometrically.

This avoids hard-coding 3×3 array assumptions.

---

90. Slope Estimation

Fit local ground plane:

```
z = ax + by + c
```

using valid neighbouring ground samples.

Slope:

```
slope =
atan(
```

```
    sqrt(a² + b²)
)
```

---

91. Roughness

Possible V1 metric:

```
R =
sqrt(
    mean(
        (zi - z_plane(xi,yi))²
    )
)
```

representing deviation from local plane.

---

92. Step Height

Estimate:

```
step =
max absolute difference
between neighbouring valid ground surfaces
```

with resolution-aware neighbourhood queries.

---

93. Hard Traversability Vetoes

Cell shall be blocked if any reliable condition holds:

```
occupied vertical obstacle

slope > vehicle max

step > vehicle max

vertical clearance insufficient

explicit non-traversable semantic class
```

---

94. Unknown State

Insufficient evidence shall produce:

```
UNKNOWN
```

not:

```
TRAVERSABLE
```

This is critical.

Unknown and blocked must remain separate states.

---

95. Soft Traversability Cost

If no hard veto:

```
C =
ws × slope_norm
+
wr × roughness_norm
+
wh × step_norm
+
wm × semantic_cost
+
wu × uncertainty
```

Then:

```
T = 1 - clamp(C,0,1)
```

---

96. Traversability Categories

Internal numerical representation:

```
0.0–1.0
```

UI representation:

```
TRAVERSABLE

CAUTION

POOR

BLOCKED

UNKNOWN
```

Thresholds SHALL be configuration values.

---

97. Rolling Map

The map SHALL maintain bounded radius.

Default:

```
100 m
```

As vehicle moves:

```
new tiles allocated

old tiles outside radius retired
```

No full-map memory copy is permitted for routine vehicle motion.

---

98. Tile Retirement

Tile may be deleted if:

```
tile distance from current vehicle
>
map_radius + retirement_margin
```

and it contains no protected debug state.

---

99. Staleness

Every cell SHALL store:

```
last_seen
```

Age:

```
age = current_time - last_seen
```

Static data decays slowly.

Dynamic data decays rapidly.

---

100. Unknown Transition

A cell exceeding:

```
static_stale_threshold
```

shall eventually transition toward:

```
UNKNOWN
```

instead of being trusted indefinitely.

---

101. ROS QoS

Raw LiDAR

Recommended:

```
SensorDataQoS

best effort

small queue
```

Reason:

Old sensor frames are less useful than recent frames.

Odometry

Recommended:

```
reliable

depth 20–50
```

Semantic cloud

Recommended:

```
best effort or reliable after profiling

depth 2
```

Metrics

```
reliable

depth 10
```

Static configuration / low-rate map metadata

```
reliable
```

---

102. Backpressure Policy

AVR-Map SHALL NOT permit unlimited sensor backlog.

Semantic inference queue:

```
maximum approximately 2 frames
```

When overloaded:

```
drop oldest unprocessed frame
```

rather than process increasingly stale perception.

---

103. Critical Pipeline Policy

Prefer:

```
latest state
```

over:

```
complete historical processing
```

for real-time autonomous perception.

---

104. Concurrency Architecture

Recommended:

```
Thread / process 1:
LiDAR + preprocessing

GPU process:
semantic inference

Thread 2:
adaptive map update

Thread 3:
tracking

Thread 4:
traversability

Thread 5:
visualisation / metrics
```

---

105. Visualisation Shall Not Block Mapping

RViz/Foxglove rendering must operate outside the critical update path.

If visualisation runs slowly:

```
map engine remains real-time
```

---

106. Map Update Frequency

Target:

```
10 Hz
```

matching typical prototype LiDAR playback.

Traversability MAY update at:

```
5–10 Hz
```

if necessary.

Visualisation MAY update at:

```
2–10 Hz
```

depending on complexity.

---

107. Performance Target

Mandatory target:

```
≥ 10 LiDAR frames/sec sustained
```

on the declared reference hardware.

Stretch:

```
15–20 FPS
```

The actual target depends on semantic inference hardware.

---

108. End-to-End Latency

For 10 Hz sensor:

```
frame period = 100 ms
```

Target:

```
p95 end-to-end
≤ 100 ms
```

or the prototype shall clearly report inability to keep real-time rate.

---

109. Stage Latency Budget

Initial engineering budget:

```
input + validation       ≤ 3 ms

transform/preprocess     ≤ 10 ms

semantic inference       ≤ 50 ms

adaptive map update      ≤ 20 ms

tracking                 ≤ 10 ms

traversability           ≤ 15 ms

metrics/publish          ≤ 5 ms
```

These are **targets**, not literature claims.

Because components can pipeline concurrently, their sums do not directly equal steady-state throughput.

---

110. Deadline Behaviour

If semantic inference exceeds frame budget:

System MAY:

```
skip selected frames

reuse last semantic result briefly

switch to ground-truth provider for mapping tests
```

It SHALL NOT build an ever-growing queue.

---

111. Benchmark Instrumentation

Every pipeline stage SHALL generate timing measurements.

Use:

```
std::chrono::steady_clock
```

for C++.

CUDA events SHOULD measure pure GPU inference time separately.

---

112. Required Timers

```
input_ms

preprocess_ms

semantic_ms

mapping_ms

tracking_ms

traversability_ms

publish_ms

end_to_end_ms
```

---

113. Metrics Message

Create:

```
avr_msgs/msg/MapMetrics.msg
```

Suggested fields:

```
std_msgs/Header header

float32 fps

float32 end_to_end_ms

float32 semantic_ms

float32 mapping_ms

float32 tracking_ms

float32 traversability_ms

uint64 active_cells

uint64 active_tiles

uint64 l0_cells

uint64 l1_cells

uint64 l2_cells

uint64 l3_cells

uint64 estimated_map_bytes

uint64 process_rss_bytes

uint64 uniform_baseline_cells

uint64 uniform_baseline_bytes

float32 cell_reduction_percent

float32 memory_reduction_percent

uint32 active_tracks

uint32 dropped_frames
```

---

114. Actual Memory Measurement

AVR-Map SHALL distinguish:

```
logical map memory

container capacity

process RSS
```

These are not the same.

At minimum report:

```
sizeof(Cell) × active allocated cells

tile/container overhead estimate

actual process RSS
```

---

115. Uniform Baseline

A uniform comparison implementation SHALL exist.

Primary baseline:

```
5 cm uniform grid
```

over identical spatial extent.

If actual hardware cannot allocate it comfortably:

```
smaller identical benchmark region
```

shall be used, while transparently reporting extent.

---

116. Fair Benchmarking Rule

Uniform and adaptive maps MUST receive:

```
same input points

same poses

same semantic results

same map extent

same test duration

same machine
```

Only map representation shall differ.

---

117. Mapping Benchmark Outputs

Required:

```
cell count

tile count

logical memory

process memory

mean update latency

p50 update latency

p95 update latency

maximum update latency

throughput
```

---

118. Semantic Preservation Metric

For every labelled point or small-object region:

Compare original semantic evidence against mapped output.

Define:

```
Small Object Preservation Rate =
safety-critical objects represented after mapping
/
safety-critical objects present in semantic input
```

Measure by distance band.

---

119. Elevation Accuracy

Where reference surface is available:

```
RMSE =
sqrt(
  mean(
    (z_est - z_gt)²
  )
)
```

Report by:

```
distance band

terrain class

resolution level
```

---

120. Adaptive Ablation Modes

Resolution controller MUST support modes:

```
MODE_A:
distance only

MODE_B:
distance + semantics

MODE_C:
distance + semantics + uncertainty

MODE_D:
distance + semantics + uncertainty + geometry + dynamics
```

This allows us to prove which mechanisms add value.

---

121. Runtime Mode Configuration

Example:

```
adaptive:
  mode: full
```

Options:

```
distance

semantic

semantic_uncertainty

full
```

---

122. ROS Visualization Outputs

Publish:

```
/avr/viz/resolution

/avr/viz/semantic

/avr/viz/elevation

/avr/viz/traversability

/avr/viz/overhang

/avr/viz/tracks
```

using appropriate:

```
visualization_msgs/MarkerArray

PointCloud2
```

```
grid_map messages
```

---

123. Resolution Visualisation

Each leaf cell shall be rendered with actual spatial footprint.

Meaning:

```
5 cm cell
visually small

40 cm cell
visually large
```

Judges must be able to see the adaptivity directly.

---

124. Derived Dense Grid

For compatibility with standard tools, AVR-Map MAY generate a temporary dense raster of:

```
elevation

traversability
```

at configurable output resolution.

This is an **output view**.

It is not the core internal representation.

---

125. grid_map Integration

`grid_map` SHOULD be used for:

```
derived layers

visualisation

filtering experiments
```

```
planner compatibility
```

but SHALL NOT replace the custom adaptive map core.

`grid_map` supports multi-layer 2.5D information such as elevation, variance, friction, normals and traversability.

---

126. Services

Required:

```
/avr/reset_map
```

using:

```
std_srvs/Trigger
```

Recommended:

```
/avr/save_snapshot

/avr/set_benchmark_mode
```

---

127. Configuration Files

```
config/
├── map.yaml
├── resolution.yaml
├── semantic.yaml
├── tracker.yaml
├── vehicle.yaml
├── traversability.yaml
├── benchmark.yaml
└── visualization.yaml
```

---

128. Example map.yaml

```
map:
  radius_m: 100.0

  base_resolution_m: 0.05

  max_level: 3

  tile_size: 16

  rolling: true

  frame_id: odom

  stale_timeout_s: 5.0

  point_replay_frames: 3
```

---

129. Example resolution.yaml

```
distance_bands:
  - max_m: 10.0
    level: 0

  - max_m: 30.0
    level: 1

  - max_m: 60.0
    level: 2

  - max_m: 100.0
    level: 3

refinement:

  semantic: true

  uncertainty: true

  dynamic: true

  geometry: true

coarsening:
```

```
  stable_frames: 10
```

---

130. Example tracker.yaml

```
tracker:

  clustering:
    method: dbscan

    eps_m: 0.7

    min_points: 4

  association:
    max_distance_m: 3.0

  lifecycle:
    confirm_hits: 3

    max_misses: 5

  motion:
    speed_threshold_mps: 0.3
```

These are starting defaults only.

---

131. Build Configuration

All C++ release benchmarks SHALL use:

```
CMAKE_BUILD_TYPE=Release
```

not debug builds.

Compiler warnings:

```
-Wall
-Wextra
-Wpedantic
```

SHOULD be enabled.

132. Repository Layout

```
AVR-Map/
│
├── README.md
├── LICENSE
│
├── docs/
│   ├── PRD.md
│   ├── TRD.md
│   ├── architecture.md
│   ├── algorithms.md
│   ├── experiments.md
│   └── demo.md
│
├── config/
│
├── ros2_ws/
│   └── src/
│       ├── avr_msgs/
│       ├── avr_input/
│       ├── avr_preprocess/
│       ├── avr_semantics/
│       ├── avr_mapping/
│       ├── avr_tracking/
│       ├── avr_traversability/
│       ├── avr_visualization/
│       ├── avr_benchmark/
│       └── avr_bringup/
│
├── models/
│
├── scripts/
│
├── datasets/
│
├── tests/
│
├── rviz/
│
├── foxglove/
│
└── results/
```

133. Mapping Library Internal Layout

Recommended:

```
avr_mapping/
│
├── include/avr_mapping/
│   ├── adaptive_map.hpp
│   ├── tile.hpp
│   ├── cell.hpp
│   ├── morton.hpp
│   ├── resolution_controller.hpp
│   ├── point_aggregator.hpp
│   ├── semantic_fusion.hpp
│   ├── vertical_profile.hpp
│   └── replay_buffer.hpp
│
└── src/
    ├── adaptive_map.cpp
    ├── tile.cpp
    ├── morton.cpp
    ├── resolution_controller.cpp
    ├── point_aggregator.cpp
    ├── semantic_fusion.cpp
    ├── vertical_profile.cpp
    └── replay_buffer.cpp
```

---

134. Unit Testing Requirements

Framework:

```
GoogleTest
```

Mandatory tests:

```
Morton encode/decode

negative coordinate indexing

cell boundary ownership

parent/child alignment

split correctness
```

```
merge correctness

no duplicate leaf ownership

point replay

stale deletion

semantic hazard preservation

vertical clearance

traversability veto

tracker state prediction
```

---

135. Boundary Tests

Explicit edge cases:

```
x = 0

x = cell boundary

x = -cell boundary

x = ±map radius

point exactly at parent boundary

point exactly at child boundary
```

Expected:

```
exactly one owning leaf
```

---

136. Split Test

Given:

```
one 20 cm parent
```

containing known points,

after split:

```
4 × 10 cm children
```

shall contain correctly re-binned observations.

Sum of replayed observations SHALL equal valid parent source observations.

---

137. Semantic Preservation Test

Construct:

```
100 road points

3 pole points
```

Expected:

```
dominant road

hazard pole retained

refinement requested
```

---

138. Overhang Test

Synthetic scene:

```
ground = 0 m

overhead plane = 1.8 m
```

Expected:

```
VALID_GROUND

OVERHANG
```

```
clearance ≈ 1.8 m
```

within tolerance.

---

139. Dynamic Layer Test

Moving object passes through road cell.

After object leaves and TTL expires:

```
dynamic occupancy removed

static road remains
```

---

140. Integration Tests

Required:

```
dataset → preprocessing

preprocessing → semantics

semantics → mapper

mapper → tracker

mapper → traversability

mapper → visualisation

complete rosbag playback
```

---

141. Complete-System Test

Command:

```
ros2 launch avr_bringup demo.launch.py
```

shall start:

```
input

preprocessing

semantics

mapping

tracking

traversability

benchmark

visualisation
```

---

142. Demo Reproducibility

At least one dataset sequence SHALL be designated:

```
golden demo sequence
```

Example:

```
SemanticKITTI sequence X
```

or controlled rosbag.

The same sequence must reproduce benchmark figures used in the SIH presentation.

---

143. Logging

Use ROS 2 logging levels:

```
DEBUG

INFO

WARN

ERROR
```

Per-frame normal operation SHALL NOT flood console.

Detailed frame diagnostics go to:

```
CSV / JSON / rosbag
```

---

144. Benchmark CSV

Each processed frame:

```
timestamp

frame_id

points_in

points_filtered

semantic_ms

mapping_ms

tracking_ms

traversability_ms

total_ms

active_cells

l0

l1

l2

l3

active_tiles

memory_bytes

tracks

dropped_frames
```

---

145. Benchmark Repetition

Performance experiments SHOULD run:

```
≥ 3 repeats
```

and report:

```
mean

median

p95

standard deviation
```

Do not report a single fastest run as system performance.

---

146. Warm-Up

GPU inference experiments SHALL exclude startup warm-up.

Recommended:

```
first 20–50 frames
```

excluded from latency statistics.

---

147. Failure Handling

Missing transform

Action:

```
drop frame

increment diagnostic counter

WARN throttled
```

Do not insert points into wrong coordinate frame.

Missing semantic output

Possible configured behaviour:

```
skip mapping

or

geometry-only fallback
```

GPU inference failure

Fallback:

```
geometry-only mapping
```

may keep demo operational.

---

148. Geometry-Only Fallback

When neural semantics fail:

AVR-Map SHOULD still support:

```
elevation

occupancy

roughness

geometry refinement

basic traversability
```

Semantic-dependent features become:

```
UNKNOWN / disabled
```

This improves prototype robustness.

---

149. Data Validation

Reject or sanitize:

```
NaN points

Inf points

unreasonable coordinates

timestamps outside tolerance

invalid semantic labels

negative confidence

confidence > 1
```

---

150. Parameter Validation

At launch:

```
r0 > 0

distance bands increasing

resolution levels dyadic

map radius > max distance band

vehicle dimensions positive

thresholds in valid ranges
```

Invalid configuration SHALL stop startup with clear error.

---

151. Determinism

Ground-truth mode with fixed configuration should be reproducible.

Any randomness in:

```
DBSCAN

sampling

model inference
```

SHOULD be controlled where possible for benchmark runs.

---

152. Dependency Isolation

Neural inference dependencies SHOULD live in a separate environment/container from the mapping core.

Reason:

SalsaNext and older research models may require older CUDA/PyTorch stacks.

The mapper should remain modern ROS 2 C++.

---

153. Container Strategy

Recommended:

```
Container A:
ROS + AVR C++ stack

Container B:
semantic inference
```

or a unified compatible container if dependency testing proves stable.

Communication remains ROS 2.

---

154. Version Pinning

The repository SHALL record:

```
Ubuntu version

ROS distro
```

```
CUDA version

PyTorch version

model commit

dataset version

compiler version
```

for reproducibility.

---

155. Security Scope

Prototype has no internet-facing control interface.

No remote actuation is performed.

Therefore security focus is:

```
dependency integrity

input validation

no arbitrary shell execution

safe file paths

reproducible containers
```

Production robotics cybersecurity is outside V1 scope.

---

156. Visualisation Performance

Visualisation MAY sample cells when necessary.

Never reduce mapping correctness simply to render more cubes in RViz.

Suggested:

```
map internal update:
10 Hz
```

```
heavy cell visualization:
2–5 Hz
```

---

157. Foxglove Dashboard

Optional but recommended panels:

```
FPS

latency timeline

memory

cell count

resolution distribution

dynamic track count

dropped frames
```

RViz remains required for spatial 3D inspection.

---

158. Acceptance Benchmark 1 — Adaptive Memory

Input:

same sequence.

Compare:

```
uniform 5 cm

vs

adaptive
```

Pass if:

```
adaptive uses fewer active cells
```

with no unacceptable loss of required near-field/safety-critical information.

No fixed percentage is mandated before measurement.

---

159. Acceptance Benchmark 2 — Resolution Behaviour

For a controlled scene:

```
near regions → fine

far flat regions → coarse

far safety-critical object → refined
```

shall be visible in map.

---

160. Acceptance Benchmark 3 — Small Object

At least one:

```
pole

cone

pedestrian

small barrier
```

shall trigger refinement compared with distance-only mode.

---

161. Acceptance Benchmark 4 — Uncertainty

Case A:

```
high uncertainty
+
adequate returns
```

→ refinement.

Case B:

```
high uncertainty
+
insufficient returns
```

→ UNKNOWN rather than fake detail.

---

162. Acceptance Benchmark 5 — Dynamics

A moving object shall:

```
receive track ID

have velocity estimate

activate dynamic layer

not permanently modify static terrain
```

---

163. Acceptance Benchmark 6 — Overhang

Controlled overhead structure shall produce:

```
ground elevation

overhead elevation

clearance estimate
```

and traversability shall respect vehicle height.

---

164. Acceptance Benchmark 7 — Rough Terrain

Rough/slope regions shall produce higher traversability cost than nearby smooth terrain when vehicle parameters remain constant.

---

165. Acceptance Benchmark 8 — Real-Time

On declared reference hardware:

```
sustained input playback = 10 Hz
```

System SHALL either:

```
process at ≥10 Hz
```

or transparently report dropped frames and bottleneck stage.

The SIH demo target is ≥10 Hz.

---

166. Acceptance Benchmark 9 — No Duplicate Ownership

Randomly sample spatial coordinates.

For each coordinate:

```
active_leaf_count == 1
```

inside valid mapped space.

---

167. Acceptance Benchmark 10 — Bounded Memory

During extended local driving:

```
map memory shall plateau
```

rather than increase indefinitely.

This proves rolling-map pruning functions correctly.

---

168. Technical Risks

R1 — Neural model dependency issues

Mitigation:

```
ground-truth provider

model container

provider abstraction
```

R2 — Adaptive map becomes overly complex

Mitigation:

```
distance-only mode first

then add one refinement source at a time
```

R3 — RViz rendering dominates runtime

Mitigation:

```
asynchronous lower-rate visualization
```

R4 — Semantic errors remove obstacles

Mitigation:

```
geometric hazard fallback

uncertainty

secondary hazard

replay refinement
```

R5 — Far-range fake detail

Mitigation:

```
density gate
```

R6 — Resolution oscillation

Mitigation:

```
hysteresis
```

---

169. Implementation Priority

Development order SHALL be:

```
1. dataset reader

2. uniform baseline

3. dyadic adaptive map

4. Morton/tile indexing

5. resolution visualisation

6. ground/elevation fusion

7. semantic ground-truth integration

8. semantic refinement

9. benchmark framework

10. small-object preservation

11. uncertainty

12. traversability

13. overhang

14. dynamic tracker

15. neural semantic provider
```

```
16. dashboard polish
```

---

170. Critical Development Rule

Do **not** start by training or modifying a large neural network.

The mapping innovation must be completed first using ground-truth semantics.

Otherwise the team risks spending most of the prototype period solving:

```
CUDA

dependencies

training

model conversion
```

instead of solving SIH26053.

---

171. Prototype Milestone M0 — Environment

Complete when:

```
Ubuntu
ROS 2
RViz
workspace
dataset
```

all run.

---

172. Milestone M1 — Baseline Mapper

Complete when:

```
LiDAR
→ uniform 5 cm map
→ RViz
```

works.

---

173. Milestone M2 — Adaptive Hierarchy

Complete when:

```
5/10/20/40 cm

distance-based hierarchy

Morton/tile indexing

rolling map
```

works.

This is the first major project milestone.

---

174. Milestone M3 — Semantic Adaptive Map

Complete when:

```
dataset semantic labels
→ safety classification
→ local refinement
→ small-object preservation
```

works.

This is the first strong SIH demonstration.

---

175. Milestone M4 — Multi-Surface Terrain

Complete when:

```
ground

roughness

step

overhang

clearance

traversability
```

are present.

---

176. Milestone M5 — Dynamic World

Complete when:

```
clustering

tracking

velocity

dynamic occupancy
```

operate.

---

177. Milestone M6 — Real AI

Complete when:

```
pretrained neural segmentation
```

replaces ground-truth labels without changing map architecture.

---

178. Milestone M7 — Final Benchmark

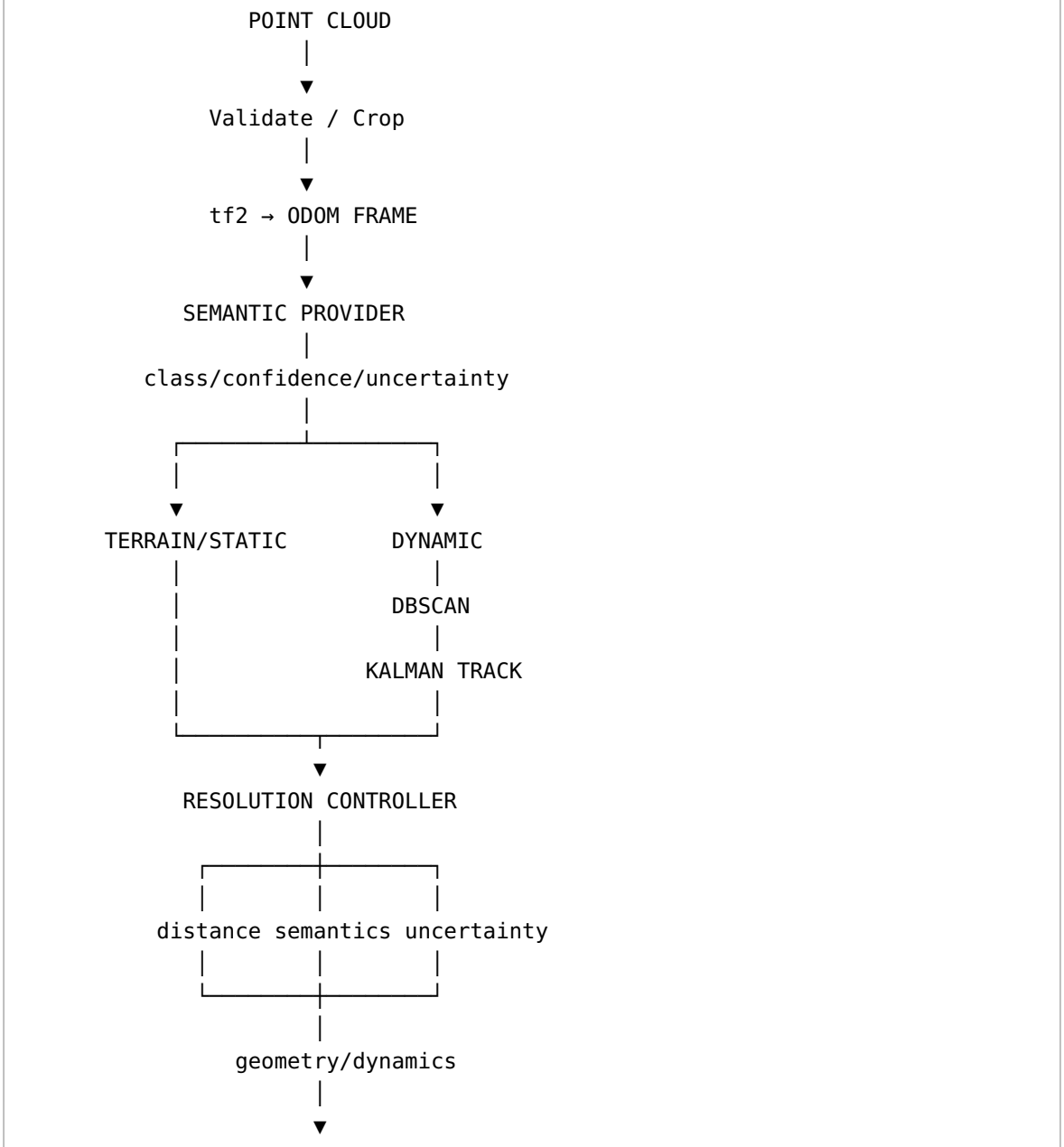
Complete when:

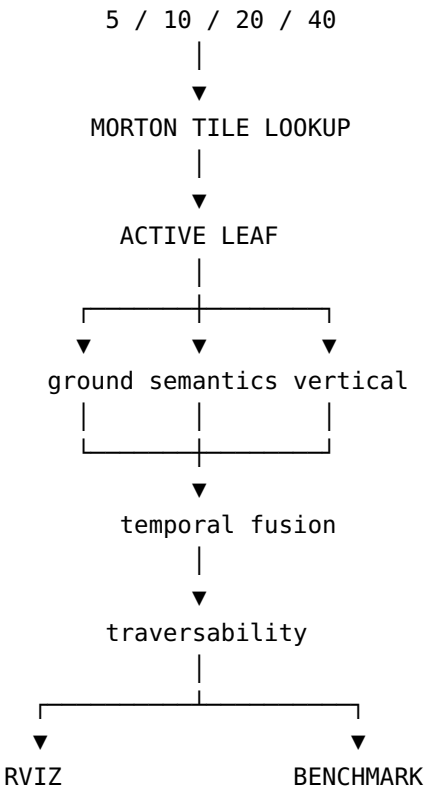
```
uniform
vs
distance-only
vs
semantic adaptive
vs
full adaptive
```

results are recorded.

---

179. Final Reference Pipeline





---

180. Final Technical Definition

AVR-Map V1 is technically defined as:

> **A ROS 2 real-time local mapping framework that receives sequential LiDAR observations, converts them to semantic and geometric evidence, stores them in a sparse Morton-indexed dyadic 2.5D hierarchy, adaptively refines regions based on distance, semantic safety relevance, uncertainty, motion and terrain complexity, preserves minority hazards and lightweight vertical structure, separates dynamic actors from persistent terrain, estimates vehicle-specific traversability, and quantitatively compares its memory and runtime performance against a uniform fine-resolution map.**

---

181. Most Important Engineering Principles

The implementation team should remember these ten rules:

Rule 1

The map is the product. The neural network is a component.

Rule 2

Distance chooses a baseline resolution; safety information may refine it.

Rule 3

Never create fake information by subdividing sparse observations.

Rule 4

One spatial location has exactly one active leaf owner.

Rule 5

Use integer dyadic geometry, not floating-point boundary decisions.

Rule 6

Never permanently erase minority safety information through majority aggregation.

Rule 7

Dynamic objects do not permanently modify the static terrain map.

Rule 8

Unknown is different from free and different from blocked.

Rule 9

Visualisation and neural inference must not be allowed to destroy mapping real-time performance.

Rule 10

Every efficiency claim must be measured against the same-input uniform baseline.

---

182. Definition of Technical Completion

The TRD is satisfied when a fresh Ubuntu machine with the documented dependencies can launch AVR-Map and reproduce:

```
Dataset / LiDAR stream
        ↓
semantic point cloud
        ↓
adaptive hierarchy
        ↓
```

```
5/10/20/40 cm cells
        ↓
semantic refinement
        ↓
small-object retention
        ↓
multi-surface elevation
        ↓
dynamic tracks
        ↓
traversability
        ↓
RViz visualisation
        ↓
uniform-vs-adaptive benchmark
```

and the run produces reproducible logs containing:

```
FPS

latency

active cells

resolution distribution

memory

semantic preservation

elevation accuracy

dynamic tracks

traversability
```

without manual changes during the demonstration.

---

183. Final V1 Technical Stack

```
OPERATING SYSTEM
Ubuntu 24.04

ROBOTICS
ROS 2 Jazzy
tf2
RViz2
```

```
rosbag2

MAPPING CORE
C++17+
Eigen
PCL

MAP REPRESENTATION
Sparse tiled linear quadtree
Morton/Z-order indexed
5/10/20/40 cm levels

SEMANTICS
GroundTruthProvider first
SalsaNext neural provider second

UNCERTAINTY
normalized entropy / model confidence

TRACKING
DBSCAN
+
Kalman Filter
+
Hungarian association

TERRAIN
probabilistic elevation
+
slope
+
roughness
+
step

OVERHANG
ground
+
lowest overhead surface
+
vertical clearance

TRAVERSABILITY
vehicle-specific
geometry + semantics + uncertainty

VISUALISATION
RViz2
+
optional Foxglove
```

```
BENCHMARK
uniform 5 cm
vs
adaptive AVR-Map
```

---

184. Technical Position We Are Locking

For the prototype, the chosen system is therefore **not**:

```
PointNet++ demo
```

and not:

```
full OctoMap clone
```

and not:

```
normal elevation map with different fixed rings
```

It is:

AVR-Map

Adaptive Semantic Multi-Surface Hierarchical 2.5D LiDAR Mapping

whose primary technical contribution is:

```
SENSOR INFORMATION
        +
SEMANTIC IMPORTANCE
        +
UNCERTAINTY
        +
GEOMETRIC COMPLEXITY
        +
DYNAMIC STATE
        ↓
HOW MUCH MAP DETAIL
SHOULD EXIST HERE?
```

That question drives the adaptive representation.

The implementation should prove that **spatial resolution itself can become an intelligent resource** rather than a globally fixed property of a robotics map.